

# 강화학습 조사자료 참고 정리

대상 자료:

- [강화학습기틀조사.pdf]

목적:

- 예나 조사자료 중 우리 발표/보고서에 가져와도 되는 부분
- 그대로 가져오면 안 되는 부분
- 우리 실험 결과 기준으로 다시 써야 하는 부분 를 구분하기 위함.

## 1. 전체 판단

해당 자료는 **강화학습 도입 배경과 기본 프레임**을 설명하는 참고자료로는 매우 유용하다. 특히 아래 부분은 우리 프로젝트와 공통점이 크다.

- LSTM은 유지하고 RL을 자원 정책 계층으로 추가
- PPO 기반 이산 행동 정책
- State / Action / Reward 구조 설명
- Sim-to-Real gap, Reward 민감도, Reactive baseline의 강함 같은 리스크 인식

다만 이 자료는 **설계 조사 단계**에 가까우며, 우리 팀은 실제로 RL 파이프라인과 Docker 검증까지 수행했기 때문에 그대로 가져오면 안 되는 부분도 분명히 존재한다.

즉 이 자료는

- 구조 설명용 참고자료로는 좋지만
- 우리 실험 결과를 대신하는 자료로 쓰면 안 된다.

## 2. 그대로 가져와도 되는 부분

아래 내용은 큰 틀에서 우리 프로젝트와 공통점이 커서, **표현만 조금 다듬어 재사용하기 좋은 부분**이다.

### 2.1 RL 도입 배경

가져와도 되는 핵심 메시지:

- 규칙 기반 Predictive-Hybrid가 Reactive를 안정적으로 넘지 못했다.
- 예측값이 조금만 빗나가도 p95 spike가 발생할 수 있다.
- 수동 파라미터 조정만으로는 결과가 흔들릴 수 있다.
- 따라서 상태와 성능 결과를 함께 보고 정책을 스스로 조정하는 RL이 대안이 될 수 있다.

우리 발표/보고서용 추천 문장:

기존 규칙 기반 Predictive-Hybrid는 일부 구간에서 평균 자원 효율 가능성을 보였지만, Reactive baseline을 안정적으로 상회하지는 못하였다. 특히 예측 오차와 수동 파라미터 조정에 따라 p95 latency와 SLA 위반 수가 크

게 흔들리는 한계가 확인되었다. 이에 따라 예측 결과와 현재 시스템 상태를 함께 고려하여 자원 할당 정책을 스스로 조정하는 강화학습 기반 접근을 다음 단계로 검토하였다.

## 2.2 LSTM + RL 역할 분리

가져와도 되는 핵심 메시지:

- LSTM은 미래 트래픽 예측 담당
- RL은 CPU / Replica 정책 결정 담당
- RL은 LSTM을 대체하는 것이 아니라, 예측 위에 얹히는 정책 계층임

우리 발표/보고서용 추천 문장:

본 연구에서 LSTM은 미래 트래픽 예측 모듈로 유지하고, 강화학습은 예측값과 현재 시스템 상태를 바탕으로 CPU 및 Replica 조정 행동을 결정하는 정책 계층으로 추가하였다. 즉 RL은 예측기를 대체하는 것이 아니라, 예측 결과를 실제 자원 할당 정책으로 연결하는 역할을 수행한다.

## 2.3 PPO 선택 이유

가져와도 되는 핵심 메시지:

- 이산 행동 공간 지원
- CPU 기반 실험 가능
- **Gymnasium** 커스텀 환경과 연결 쉬움
- 학습 로그 관리 용이

우리 발표/보고서용 추천 문장:

강화학습 알고리즘으로는 PPO를 선택하였다. PPO는 이산 행동 공간을 직접 지원하므로 **HOLD**, **CPU\_UP**, **CPU\_DOWN**, **REP\_UP**, **REP\_DOWN**과 같은 자원 제어 행동을 비교적 자연스럽게 모델링할 수 있다. 또한 **Gymnasium** 기반 커스텀 환경과 연동이 용이하고, CPU 환경에서도 반복 학습 실험이 가능하다는 장점이 있다.

## 2.4 예상 리스크 인식

가져와도 되는 핵심 메시지:

- Sim-to-Real gap
- reward 설계 민감도
- Reactive baseline이 여전히 강함

우리 발표/보고서용 추천 문장:

강화학습 적용 시 주요 리스크는 세 가지로 정리할 수 있다. 첫째, 시뮬레이터에서 학습한 정책이 실제 Docker 실행 환경에서 동일하게 동작하지 않을 수 있는 sim-to-real gap이 존재한다. 둘째, reward 설계에 따라 정책이 최소 자원 유지 또는 최대 자원 유지의 극단으로 수렴할 가능성이 있다. 셋째, Reactive baseline이 이미 강한 성능을 보이기 때문에 RL 정책이 이를 안정적으로 상회하려면 보다 정교한 학습 설계가 필요하다.

## 3. 그대로 가져오면 안 되는 부분

아래는 우리 결과와 정확히 일치하지 않거나, 현재 단계에서 그대로 쓰면 오해를 부를 수 있는 부분이다.

### 3.1 SLA 정의를 $p95 > 1000ms$ 로 쓰는 표현

예나 조사자료 표현:

- $p95 > 1000ms$ 이면 -10

문제점:

- 우리 실험에서 `sla_violation_count`는 개별 요청 `latency > 1000ms` 기준으로 집계했음
- 즉 우리 기준은  $p95$  기준 SLA가 아니라 요청 단위 SLA 위반 개수

따라서 우리 문서에서는 아래처럼 써야 한다.

추천 표현:

본 실험에서 SLA 위반은 개별 요청 응답시간이 1000ms를 초과하는 경우로 정의하였다.

### 3.2 Reward 함수 예시를 그대로 사용하는 것

예나 조사자료 reward 예시:

- $p95 > 1000ms \rightarrow -10$
- `cpu_quota, replicas` 비용
- $avg\_latency < 500$  보너스

문제점:

- 우리는 실제로 1차~5차 RL 실험을 하며 reward를 여러 번 바꿔 봤고,
- 단순 reward로는
  - 최소 자원 collapse
  - 최대 자원 collapse 가 반복된 것을 이미 확인했음

즉 우리 자료에서는 단순 reward 예시를 그대로 쓰기보다, **“실제 구현 과정에서 reward 설계가 매우 민감했고 극단 정책 수렴 문제가 발생했다”**는 점을 같이 써야 한다.

추천 표현:

초기에는 SLA 위반, fail, latency, 자원 사용량을 단순 가중합한 reward를 사용하였으나, 실제 학습 결과 정책이 최소 자원 유지 또는 최대 자원 유지의 극단으로 반복 수렴하였다. 따라서 reward는 단순 예시 수준이 아니라, 품질과 효율의 균형을 학습시키기 위한 핵심 설계 요소로 확인되었다.

### 3.3 “GPU 없이 수십만 step을 수 시간 내 학습” 같은 낙관적 표현

문제점:

- 이론적으로는 가능하더라도, 우리 환경에서는
  - PyTorch 실행 환경 문제
  - simulator 품질 문제
  - 실제 Docker 검증 반복 비용 등이 존재했음

따라서 너무 낙관적으로 쓰기보다, 아래처럼 현실적으로 정리하는 게 좋다.

추천 표현:

학습 자체는 시뮬레이터 기반으로 반복 수행할 수 있으나, 실제 정책 품질은 reward 설계, simulator 정확도, 그리고 이후 Docker 검증 결과까지 함께 고려하여 판단해야 한다.

### 3.4 “RL이 Reactive보다 더 좋아질 것이다”라는 뉘앙스

문제점:

- 우리는 실제 실험에서 RL이 아직 Reactive를 넘지 못했음
- 현재 RL은 구조 검증과 탐색 단계에 가까움

따라서 발표/보고서에서는

- “Reactive를 이긴다”가 아니라
- “규칙 기반 한계를 넘기 위한 다음 단계로 RL을 검토했다” 로 쓰는 게 맞다.

추천 표현:

현 단계의 RL은 Reactive baseline을 능가하는 완성형 정책이라기보다, 규칙 기반 자원 할당 정책의 한계를 넘기 위한 다음 단계 탐색으로 해석하는 것이 적절하다.

## 4. 우리 기준으로 다시 써야 하는 부분

아래는 예나 조사자료를 그대로 인용하기보다, 우리 실험 결과를 반영해 새로 정리해야 하는 핵심 항목이다.

### 4.1 State

예나 조사자료:

- 8개 변수

우리 쪽은 실제로 더 확장된 state를 써봤음.

예:

- current\_rps
- predicted\_rps
- lookahead\_peak\_rps
- cpu\_usage\_pct
- avg\_latency\_ms
- p95\_latency\_ms
- fail\_count
- sla\_violation\_count
- current\_cpu
- current\_replicas
- rps\_gap
- delta\_actual\_rps

- `delta_predicted_rps`
- `delta_p95_latency`

즉 발표에선 “기본 구상은 8개였지만, 실제 실험 과정에서 추세와 피크 정보를 더 반영하도록 state를 확장하였다” 라고 정리하는 게 좋다.

추천 표현:

초기 설계에서는 현재 RPS, 예측 RPS, CPU 사용률, latency, fail rate, 현재 자원 상태 등 기본 8개 변수를 중심으로 상태를 구성하였다. 이후 실제 실험 과정에서는 피크 직전 선제 대응을 위해 `lookahead_peak_rps`, `rps_gap`, `delta_actual_rps`, `delta_p95_latency`와 같은 추세 변수를 추가하여 상태를 확장하였다.

## 4.2 Action

예나 조사자료:

- `HOLD`, `CPU_UP`, `CPU_DOWN`, `REP_UP`, `REP_DOWN`

우리 쪽은 이 기본 행동을 바탕으로

- `CPU_AND_REP_UP`
- `DOWN_TO_BASELINE` 같은 변형도 실험했음

즉:

- “기본 action 설계는 동일”
- “실험 과정에서 단순화/재확장을 반복” 으로 쓰는 게 맞다.

추천 표현:

기본 action space는 `HOLD`, `CPU_UP`, `CPU_DOWN`, `REP_UP`, `REP_DOWN`의 5개 이산 행동으로 시작하였으며, 이후 실험 과정에서는 정책 수렴 특성을 확인하기 위해 `CPU_AND_REP_UP`과 같은 결합 행동을 추가하거나 다시 분리하는 방식의 변형을 수행하였다.

## 4.3 Reward

이 부분은 예나 조사자료보다 훨씬 더 조심해서 써야 한다.

우리 실험에서 확인한 핵심:

- reward가 조금만 바뀌어도 정책이 크게 흔들림
- 1차/4차/5차는 최소 자원 collapse
- 2차/3차는 최대 자원 collapse

추천 표현:

Reward는 품질(SLA 위반, fail, avg/p95 latency)과 자원 비용(CPU, replica)을 함께 반영하는 가중합 형태로 설계하였다. 그러나 실제 1차~5차 실험 결과, reward 계수와 보조 shaping 항목에 따라 정책이 최소 자원 유지 또는 최대 자원 유지의 극단으로 반복 수렴하는 현상이 확인되었다. 이는 강화학습 도입에서 reward 설계가 성능 자체만큼이나 중요한 핵심 변수임을 보여준다.

## 4.4 RL 결과 해석

이 부분이 가장 중요하다.

예나 조사자료는 “이렇게 하면 될 것 같다”에 가깝고, 우리는 “실제로 해보니 어떤 문제가 생겼는지”까지 가지고 있어.

우리 기준으로 반드시 들어가야 하는 내용:

- RL 구조 연결은 성공
- PPO 학습/계획 생성/Docker 검증 흐름은 동작
- 하지만 정책은 반복적으로 극단 수렴
- 실제 품질은 fallback 개입에 많이 의존
- 따라서 baseline imitation / simulator 보정 / reward 재설계가 필요

추천 표현:

실제 구현 결과, RL 파이프라인과 Docker 검증 연결 자체는 성공하였다. 그러나 정책은 최소 자원 유지 또는 최대 자원 유지의 극단으로 반복 수렴하였고, 실제 Docker 검증에서는 HybridController fallback이 성능을 상당 부분 방어하는 구조가 나타났다. 따라서 현재 단계에서는 RL 정책이 baseline을 대체할 수준이라고 보기 어렵고, 향후에는 reward 재설계뿐 아니라 baseline imitation 및 simulator 보정이 필요하다.

## 5. 발표/보고서에 바로 가져갈 수 있는 구성

가져와도 되는 슬라이드/문단 주제

| 주제                          | 사용 가능 여부 | 비고                    |
|-----------------------------|----------|-----------------------|
| 왜 RL이 필요한가                  | 사용 가능    | 우리 실험 결과와 연결해서 재작성    |
| LSTM + PPO 구조               | 사용 가능    | 거의 동일 구조              |
| PPO 선택 이유                   | 사용 가능    | 문구만 우리 상황에 맞게 다듬기     |
| Sim-to-Real, reward 민감도 리스크 | 사용 가능    | 오히려 우리 실험이 더 강한 근거 제공 |

그대로 가져오면 안 되는 주제

| 주제                           | 사용 주의 이유           |
|------------------------------|--------------------|
| $p95 > 1000ms = SLA$         | 우리 실험 정의와 다름       |
| 단순 reward 함수 예시              | 우리 실제 실험 결과와 맞지 않음 |
| RL이 Reactive보다 더 좋아질 것이라는 전제 | 우리 결과로는 아직 입증 안 됨  |
| 학습 난이도/속도에 대한 낙관적 서술         | 우리 환경에선 보수적으로 써야 함 |

## 6. 최종 정리

이 자료는 강화학습 도입 배경과 기본 프레임을 설명하는 참고자료로는 충분히 활용 가능하다. 특히 아래 메시지는 우리 발표에도 잘 맞는다.

- 규칙 기반 Predictive-Hybrid의 한계
- LSTM 예측 유지 + RL 정책 계층 추가
- PPO 기반 이산 행동 정책
- sim-to-real gap과 reward 민감도 문제

다만 우리 팀은 이미 실제 구현과 Docker 검증까지 수행했기 때문에, 아래 부분은 반드시 우리 실험 기준으로 다시 정리해야 한다.

- SLA 정의
- state/action/reward의 실제 구현 버전
- RL 1차~5차 실험에서 나타난 극단 수렴 문제
- fallback 의존도
- 향후 imitation learning / simulator 보정 필요성

---

## 7. 한 줄 결론

예나 조사자료는 “왜 RL을 붙이려 했는가”를 설명하는 참고자료로는 매우 좋고, 우리 자료는 그 위에 “실제로 붙여보니 어떤 문제가 생겼는가”를 보여주는 확장판으로 정리하는 게 가장 자연스럽다.